

Hermes EVM Gasless Contract Audit



September 7, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Execution Flows and Authorization	6
Security Model and Trust Assumptions	6
Privileged Roles	7
Low Severity	9
L-01 ERC-777 tokensReceived Hook Is Unreachable Without ERC-1820 Registration	9
L-02 Signed Batches Invoking useNonce Consume Multiple Nonces and Invalidate Pending Signatures	10
L-03 Try-Mode REQUIRED Calls Can Be Skipped by a Preceding Break Policy	10
L-04 Try-Mode Batches Can Be Reverted Through Return-Bombing	11
L-05 Re-Delegation Does Not Reliably Invalidate Outstanding Signatures	12
L-06 Raw ECDSA, EIP-191, And EIP-2098 Compact Signatures Are Rejected	14
Notes & Additional Information	15
N-01 Try-Mode Per-Call Policies Are Not Visible To The Human Signer	15
N-02 Signed-Batch Nonces Are Consumed Only On Successful Execution	16
N-03 Zero-Address Targets Are Signed Verbatim but Executed as Self-Calls	17
N-04 Non-Explicit Imports Are Used	18
N-05 Missing Named Parameters in Mapping	18
N-06 Missing Security Contact	18
N-07 Magic Numbers In The Code	19
N-08 Inconsistent Use of Named Return Variables	20
N-09 Signed Batch Execution Is Not Observable From Event Logs	20
N-10 Misleading, Incomplete, or Missing Documentation	21
N-11 Native Value Attached to a Signed Batch Is Silently Spendable on the Relayed Path	24
Conclusion	25
Appendix	26
Issue Classification	26

Summary

Type	Account Abstraction	Total Issues	17 (14 resolved, 1 partially resolved)
Timeline	From 2026-08-14 To 2026-08-19	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	6 (5 resolved, 1 partially resolved)
		Notes & Additional Information	11 (9 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [tonkeeper/Hermes](#) repository at the [0423b8a](#) commit.

In scope were the following files:

```
contracts
├── HermesBase.sol
├── interfaces
│   ├── IERC7779.sol
│   └── IHermesNonce.sol
└── v1
    ├── HermesDelegateV1.sol
    └── HermesV1.sol
```

Update: All resolutions and the final state of the audited codebase mentioned in this report are contained at commit [63a4bd7](#).

System Overview

Hermes is a minimal smart-account system for delegated EOAs built by Tonkeeper. Using EIP-7702, an externally owned account delegates its code to the `HermesDelegateV1` implementation and thereby gains smart-account capabilities such as call batching, gas abstraction through ERC-4337, typed-data authorization, and ERC-1271/ERC-7739 signature verification, while keeping its original address and an EOA-like signature lifecycle. The system targets the canonical ERC-4337 EntryPoint versions 0.8 and 0.9.

The audited codebase consists of three contracts and two interfaces:

- The `HermesDelegateV1` contract is the EIP-7702 delegate implementation. It acts as an ERC-4337 account through `validateUserOp`, exposes the ERC-7821 minimal batch executor `execute` as its single execution entry point, verifies contract signatures through ERC-1271 with ERC-7739 defensive rehashing, and publishes its EIP-712 signing domain through ERC-5267.
- The abstract `HermesBase` contract provides the token-receiver callbacks for ERC-721, ERC-1155, ERC-777, and ERC-677 transfers, along with ERC-165 introspection and the ERC-7779 account-introspection surface used by wallets that re-delegate already-delegated accounts.
- The `HermesV1` contract is a singleton nonce manager shared by every Hermes-delegated account. It maintains a per-account monotonic counter, keyed by `msg.sender`, which serves as the sole source of replay protection for the signature-based execution path. Each delegate pins the manager's address as an immutable dependency through the minimal `IHermesNonce` interface.

A distinguishing design choice is that the delegate holds no mutable storage of its own. The only mutable state in the system is the per-account nonce inside the `HermesV1` singleton, so re-delegating an EOA to different code never leaves stale storage behind in the account.

Execution Flows and Authorization

All execution flows through the single ERC-7821 entry point `execute(bytes32 mode, bytes executionData)`, which supports two single-batch modes onto which three authorization schemes are mapped:

- In the no-`opData` mode, the batch is authorized by `msg.sender`, which must be either the account itself (the delegated EOA sending a transaction to itself, replay-protected by the EOA's own transaction nonce) or one of the two trusted EntryPoints. In the ERC-4337 flow, the `validateUserOp` function checks the delegated EOA's signature over the EIP-712-typed `userOpHash` and rejects any user operation whose calldata does not target `execute`, relying on the EntryPoint protocol's guarantee that validation runs before execution.
- In the `opData` mode, the caller is arbitrary and the batch is authorized by the delegated EOA's EIP-712 signature over an `Execute(bytes32 mode, Call[] calls, uint256 nonce, uint256 deadline)` struct carried inside `opData`. The signature is replay-protected by the Hermes manager nonce, which is consumed before any external call is made, and is time-bounded by the signed `deadline`, where a value of zero opts out of expiry.

Both modes accept the default atomic exec type, in which the first failing call reverts the whole batch, and the ERC-7579 "try" exec type, in which the otherwise-unused mode payload carries a 2-bit per-call outcome policy (OPTIONAL, REQUIRED, BREAK_ON_FAIL, or BREAK_ON_SUCCESS) governing how each call's success or failure affects the rest of the batch. Because the exec type and the policy bits are part of `mode`, and `mode` is bound into the signed digest, a submitter can neither replay a signature under a different exec type nor downgrade any call's policy.

Security Model and Trust Assumptions

The delegated EOA's private key is the sole authority over the account. There is no recovery mechanism, no guardian, and no secondary signer, so compromise of that key implies full compromise of the account. In addition, as is inherent to EIP-7702, the key can at any time re-delegate the account to arbitrary code, which means the guarantees described in this report only hold while the account remains delegated to `HermesDelegateV1`.

During the audit, the following trust assumptions were made:

- The two canonical EntryPoint contracts, version 0.8 and version 0.9, are hardcoded as trusted callers and are assumed to be correct, to exist at the same addresses on every chain where Hermes is deployed, and to uphold the ERC-4337 protocol guarantee that `validateUserOp` runs before the corresponding execution call. Each of these EntryPoints binds its own address as the EIP-712 `verifyingContract` of the `userOpHash`, which the system relies on to prevent cross-EntryPoint replay despite their independent nonce stores.
- The `HermesV1` manager address is a security-critical deployment parameter. The delegate pins it as an immutable dependency, and the manager itself has no owner, no upgrade path, no pause functionality, and no `selfdestruct`, so its correctness and identity must be verified before any delegate referencing it is shipped.
- Signatures for the `opData` path follow an EOA-like lifecycle. A signature with a `deadline` of zero never expires on its own and remains valid until its nonce is consumed, so users are expected to understand that cancelling a pending signature requires advancing the nonce, either by executing another signed batch or by calling `useNonce` directly.
- Re-delegation safety relies on the EIP-712 domain using the implementation address as its `salt`, so that re-delegating the EOA to a different implementation invalidates all outstanding signatures. Similarly, cross-account and cross-protocol replay of ERC-1271 signatures is prevented by ERC-7739 nested rehashing, which binds every verified signature to this account's domain.
- The system builds on OpenZeppelin's `ECDSA`, `MessageHashUtils`, `ERC7739Utils`, and `ERC7579Utils` libraries, which are assumed to be correct.

Furthermore, the delegate's `fallback` function accepts calls with unknown selectors without reverting in order to preserve EOA-like behavior. Integrators should be aware that a call to the account with an arbitrary selector will silently succeed rather than revert.

Privileged Roles

The system is intentionally free of administrative roles. Neither the `HermesDelegateV1` implementation nor the `HermesV1` nonce manager has an owner, an upgrade mechanism, or any pausable functionality, and the delegate's only configuration is the immutable manager address set at construction time.

The actors with distinguished capabilities are the following:

- The delegated EOA has full control over its own account. It can execute arbitrary batches by calling `execute` on itself, sign user operations for the ERC-4337 flow, sign EIP-712 `Execute` batches for relayed execution, and cancel pending signatures by advancing its nonce in the manager.
- The trusted EntryPoints (version 0.8 at `0x4337084D9E255Ff0702461CF8895CE9E3b5Ff108` and version 0.9 at `0x433709009B8330FDa32311DF1C2AFA402eD8D009`) may call `execute` in the no-`opData` mode and are the only permitted callers of `validateUserOp`, but they can only trigger execution of user operations that carry a valid signature from the delegated EOA.
- Relayers are permissionless. Anyone may submit a signed `opData` batch, but a relayer cannot alter the calls, mode, policies, nonce, or deadline that the EOA signed, and its influence is limited to choosing whether and when to submit within the signed deadline.

Low Severity

L-01 ERC-777 `tokensReceived` Hook Is Unreachable Without ERC-1820 Registration

The `HermesBase` contract implements a no-op `tokensReceived` function, and the [README](#) lists ERC-777 among the supported token-receiver callbacks. However, the ERC-777 standard requires a contract recipient to be registered as the implementer of the `ERC777TokensRecipient` interface in the ERC-1820 registry before a token will invoke this hook. Since an EIP-7702-delegated account has code, ERC-777 tokens treat it as a contract recipient, look up its implementer in the registry, find none, and revert the transfer. As the codebase never registers the account in the ERC-1820 registry, standard ERC-777 transfers to a Hermes account revert, the `tokensReceived` function is unreachable, and the documented ERC-777 support is misleading. Note that the ERC-721, ERC-1155, and ERC-677 callbacks are unaffected, as those are invoked directly by the token without the ERC-1820 indirection.

Consider documenting that a Hermes account must first register itself in the ERC-1820 registry (for example, by batch calling `setInterfaceImplementer` via `execute`, which succeeds because self-registration bypasses the `canImplementInterfaceForAddress` check) before it can receive ERC-777 tokens, or removing the ERC-777 claim from the documentation.

Update: Resolved in [pull request #10](#). The team stated:

- The [README](#) no longer lists ERC-777 among the supported token-receiver callbacks.
- The `tokensReceived` NatSpec states that the hook is unreachable until the account registers itself as its own `ERC777TokensRecipient` in the ERC-1820 registry, and gives the call that does it.

The no-op hook itself is unchanged.

L-02 Signed Batches Invoking `useNonce` Consume Multiple Nonces and Invalidate Pending Signatures

The `__verifySignedBatch` function consumes exactly one nonce per signed batch by calling `HermesV1.useNonce`, which increments the nonce of its `msg.sender`. However, `__executeBatch` then performs the batch's calls with unrestricted targets, and during execution, the account itself is the caller. Consequently, a batch that includes a call to `manager.useNonce()` advances the account's own nonce a second time, so a single signed batch containing K such calls consumes $K + 1$ nonces rather than one.

As every skipped nonce value becomes permanently unusable, any signature the account holder has pending at a skipped value (for example, a pre-signed batch handed to a relayer) can never execute. A malicious dApp can induce this by having the user sign a batch that includes an innocuous-looking call to the manager, which the wallet renders only as an opaque `data` field, thereby cancelling the user's other pending signatures without their awareness. In addition, any relayer or integration that assumes each executed batch increments the nonce by exactly one will desynchronize its nonce tracking.

Consider rejecting batch calls whose target is the manager within `execute`, which preserves the self-service cancellation primitive of calling `useNonce` directly while closing the third-party-reachable path. Alternatively, consider documenting that a signed batch may consume more than one nonce and that integrators must not rely on a one-nonce-per-batch invariant.

Update: Resolved in [pull request #11](#). The team stated:

`execute`, `HermesV1.useNonce` and the README now state that a signed batch consumes at least one nonce, and that integrators must read `nonceOf(account)` on-chain rather than track it incrementally.

L-03 Try-Mode `REQUIRED` Calls Can Be Skipped by a Preceding Break Policy

The try-mode documentation of the `HermesDelegateV1` contract describes the `REQUIRED` policy as suited for "a relayer-fee transfer that must never be skipped". However, the `__executeBatch` function only enforces `REQUIRED` as `revert-on-failure when the call is reached`, while reachability is governed independently by the `break policies`. A `BREAK_ON_SUCCESS` or `BREAK_ON_FAIL` call that triggers earlier in the batch ends

execution before a later `REQUIRED` call, which is then silently skipped while the batch still succeeds.

This exposes relayers of signed batches to fee griefing. A relayer that accepts a batch because it contains a `REQUIRED` fee transfer to itself can be presented with `[action = BREAK_ON_SUCCESS, fee = REQUIRED]`: the user's action executes, the break ends the batch, the fee call is never reached, and the transaction succeeds with the nonce consumed. The relayer pays gas and receives nothing, while the user's action ran for free.

Consider documenting that `REQUIRED` only guarantees a revert if the call fails when reached and does not guarantee execution, since a preceding break-policy call can skip it. In addition, consider advising relayer integrations to require that their fee call be unconditionally reachable, for example, by placing it at index 0 or by rejecting any batch in which a break-capable call precedes it.

Update: Resolved in [pull request #15](#). The team stated:

- `POLICY_REQUIRED` is renamed to `POLICY_REVERT_ON_FAIL`. The encoding `(01)` is unchanged, so no signature, mode or ABI is affected.
- The contract docstring and `_executeBatch` state that a policy is consulted only for a call that is reached, and that a relayer wanting an unconditional fee places it at index 0.
- New test: the action succeeds under `BREAK_ON_SUCCESS`, the `REVERT_ON_FAIL` fee at index 1 is never reached, `BatchInterrupted(0)` is emitted and the transaction succeeds.

L-04 Try-Mode Batches Can Be Reverted Through Return-Bombing

The `HermesDelegateV1` contract implements ERC-7821 batching with a try mode in which failed calls whose policy is not `POLICY_REQUIRED` are intended to be non-fatal and are only reported through the `ERC7579TryExecuteFail` event.

However, the `_executeBatch` function performs an unconstrained external call that copies the full return data into memory and then emits it as an unbounded bytes payload. A malicious or incompatible callee can revert with very large return data, forcing high gas consumption both when the return data is copied into memory and when the event is emitted. This can cause an out-of-gas condition that reverts the entire transaction, defeating try-mode

semantics by rolling back earlier successful calls. For signed batches, the revert also rolls back the nonce consumed in `_verifySignedBatch`, leaving the signature replayable.

Consider bounding attacker-controlled return data before allocating or logging it, for example by capping the copied return data at a fixed size and emitting only a truncated prefix, a `keccak256` hash, or the length. In addition, consider using an assembly call pattern that copies at most a fixed number of bytes of return data so that try mode remains non-reverting under adversarial revert data sizes.

Update: Resolved in [pull request #16](#). The team stated:

- The batch's call is now a raw `call` that copies no return data.
- A non-fatal try-mode failure is logged as `CallFailed(uint256 index)`, replacing `ERC7579TryExecuteFail(index, bytes)`.
- A fatal failure still bubbles the callee's raw revert data.
- New `ReturnBomber` mock and a regression test: a 500 KB bomb as an OPTIONAL call inside a try batch under a 1M gas limit, asserting the batch completes. Verified to fail against the audited commit with "Transaction ran out of gas".
- A fatal failure still bubbles the callee's raw revert data unbounded. The transaction reverts either way, so nothing partial survives and the only cost is the submitter's gas. The mitigation belongs to the submitter: simulate the batch before relaying it, where an oversized revert shows up as the gas it costs, and cap the transaction's `gasLimit`, which is the hard ceiling on what a hostile target can spend.

L-05 Re-Delegation Does Not Reliably Invalidate Outstanding Signatures

The [README](#) states that "re-delegating the EOA to different code invalidates all outstanding signatures", and the `_domainSeparator` docstring repeats this guarantee. However, neither signing surface fully upholds it.

On the ERC-4337 surface, `validateUserOp` accepts a `UserOperation` when the delegated EOA has signed the bare `userOpHash`, as checked by `_validateSignature`, so the signature domain contains no reference to the delegated implementation. Since an EIP-7702 authority keeps the same address across re-delegations, a `UserOperation` signed while the account was delegated to implementation A remains valid after re-delegation to an

implementation B that still recovers the signer as `address(this)` and preserves the `execute(bytes32,bytes)` selector required by the `selector check` but interprets modes or `executionData` differently in `execute`. A withheld `UserOperation` can, therefore, later be submitted by any bundler and executed with semantics different from those in effect at signing time.

On the EIP-712 surface, `_domainSeparator` does mix `delegateAddress` into the domain as `salt`, but the salt is a domain check evaluated against the delegation in effect at execution time, not a one-way cancellation. As the nonce lives in the shared `HermesV1` manager keyed by the account and persists across re-delegations, a signed `opData` batch whose deadline has not passed and whose nonce is unconsumed validates again if the authority later re-delegates back to the Hermes implementation. A user who hands out a signed batch and re-delegates, believing per the documentation that this cancels the authorization, is only protected while the account remains delegated to other code, and the only permanent cancellation is advancing the nonce via `useNonce`.

Consider binding ERC-4337 signature verification to the delegated implementation, for instance, by verifying signatures over `keccak256(abi.encode(userOpHash, delegateAddress))` so that re-delegation invalidates previously signed `UserOperations`. In addition, consider updating the `_domainSeparator` docstring and the README to state that the implementation-address salt only suspends outstanding signatures while the EOA is delegated to other code, and consider having wallet integrations expose nonce consumption, rather than re-delegation, as the cancellation flow for pending signatures.

Update: Partially Resolved in [pull request #12](#). The team stated:

- The README and the `_domainSeparator` docstring now say re-delegation suspends outstanding signatures while the account is delegated to other code, and name the permanent cancellations: spending the nonce, or an expired deadline.
- The `validateUserOp` docstring states that the signature covers the bare `userOpHash`, which does not commit to the account's code, so a withheld `userOp` survives re-delegation.
- Guidance for wallets: implement "cancel a pending signature" as a nonce-spending transaction, not as a re-delegation.

L-06 Raw ECDSA, EIP-191, And EIP-2098 Compact Signatures Are Rejected

The `isValidSignature` function only accepts the two ERC-7739 nested signature forms, verified by `_isValidTypedDataSig` and `_isValidPersonalSig`. A plain 65-byte ECDSA signature over `hash` produced by the account's own key, as well as a standard EIP-191 `personal_sign` signature, both return `0xffffffff` even though they are cryptographically valid. Since most wallets are not ERC-7739-aware, verifiers that route contract accounts to ERC-1271 (e.g., Sign-In-With-Ethereum flows, marketplaces, and Permit2-style integrations) will fail to validate signatures from a delegated account, making it strictly less signature-compatible than the bare EOA it replaced. Notably, the cross-account replay that ERC-7739 nesting defends against is weaker under EIP-7702, where a raw signature recovering to `address(this)` in `_validateSignature` is already intrinsically bound to the unique signing EOA.

In addition, `_validateSignature` relies on OpenZeppelin's `ECDSA.tryRecover(bytes32, bytes memory)`, which since version 5.0 accepts only 65-byte `(r, s, v)` signatures. A 64-byte EIP-2098 compact signature is therefore rejected on every signing surface: the signed-batch `execute` path reverts with `InvalidSignature`, `validateUserOp` returns a validation failure, and `isValidSignature` returns `0xffffffff`. This behavior is fail-closed, but wallets, relayers, and SDKs that emit compact signatures as a calldata-saving optimization will have cryptographically valid signatures rejected.

Consider, if the strict behavior is intentional, documenting prominently in the README and integration guide that Hermes accounts require ERC-7739-aware signing and accept only 65-byte ECDSA signatures on all signing surfaces. Otherwise, consider adding a fallback in `isValidSignature` that accepts a raw ECDSA signature recovering to `address(this)` (noting that this re-exposes the cross-protocol replay of a signature over a given `hash` that ERC-7739 nesting was adopted to prevent), and routing 64-byte signatures in `_validateSignature` through the `ECDSA.tryRecover(bytes32, bytes32, bytes32)` EIP-2098 overload.

Update: Resolved in [pull request #13](#). The team stated:

- `_validateSignature` splits the signature with `ECDSA.parse` and recovers from `(v, r, s)`, so the 64-byte EIP-2098 encoding is accepted on all three signing surfaces. Any other length parses to `(0, 0, 0)` and fails closed.

- `isValidSignature` falls back to a plain ECDSA signature over `hash`, checked after both ERC-7739 nested forms and the detection probe.
- The README documents the accepted signature encodings.
- Tests: raw 65-byte and 64-byte accepted, a stranger's rejected in both encodings, and a compact signature accepted on the `opData` path and by `validateUserOp`.

Notes & Additional Information

N-01 Try-Mode Per-Call Policies Are Not Visible To The Human Signer

In try mode, each call's outcome is governed by a 2-bit policy (`OPTIONAL`, `REQUIRED`, `BREAK_ON_FAIL`, or `BREAK_ON_SUCCESS`) packed least-significant-bit-first into the ERC-7821 `mode` payload, which the `executeBatch` function reads as `(policies >> (2 * i)) & 3`. Since the `verifySignedBatch` function binds the full `mode` into the EIP-712 digest, the on-chain interpretation always matches the signed bytes and the contract remains internally consistent.

The concern is at the signing boundary. Although the `opData` path is presented as fully-typed signing, this holds only for the `calls` array and not for the policies governing it, because those policies live inside the `bytes32 mode` field that wallets render as opaque hex. A signer can therefore read what a batch does but cannot verify whether each call is required, optional, or a break point, even though that property decides whether the batch is atomic or best-effort. Since per-call policies are a Hermes-specific extension with no on-chain helper to parse a `mode` back into its policy array, integrators must reimplement this bespoke layout from prose with no way to check it, and because unset slots mean `OPTIONAL`, a packing mistake fails open: a call intended to be `REQUIRED` silently becomes `OPTIONAL` and the batch continues past its failure.

Consider exposing a view that decodes a `mode` into its per-call policy array so integrators can round-trip their encoding against the contract before requesting a signature, and consider

publishing a canonical reference encoder alongside the contract so integrations do not reimplement the bit layout by hand.

Update: Resolved in [pull request #14](#). The team stated:

Added `decodeCallPolicies(bytes32 mode, uint256 callCount)` returning `uint8[] policies`: a pure mirror of what `_executeBatch` reads, so an integration can round-trip its own encoder against the contract before requesting a signature. What `execute` would reject an unsupported mode, or a try batch above the payload's 88 policy slots comes back as an empty array rather than a revert.

Tests: round-trip in both modes, unset slots read as `OPTIONAL`, and an empty array for four unsupported modes and for an 89-call try batch.

N-02 Signed-Batch Nonces Are Consumed Only On Successful Execution

The signature-authorized `opData` branch of `execute` advances the Hermes nonce inside `_verifySignedBatch`, which folds the value returned by `HermesV1.useNonce` into the signed digest before the batch runs. Because that increment and the subsequent `_executeBatch` call share a single transaction, a mined transaction whose batch reverts rolls the increment back and leaves both the nonce and the signature valid. These reverts are reachable through ordinary paths, since the default exec type bubbles up the first failing call and the try exec type does the same for a failing call carrying the `POLICY_REQUIRED` policy.

Consequently, a signed batch spends its nonce only when it runs to completion. This diverges from ordinary EOA behavior, in which a mined transaction consumes its nonce whether it succeeds or reverts: the `opData` path is labelled "EOA-like", yet the documented analogy covers only invalidating a pending signature and is silent on the mined-revert case. A batch that reverts on a transient condition, such as a temporarily insufficient balance or an intermittently reverting target, therefore stays executable once that condition clears, and since a `deadline of 0 disables expiry` as the default, that window is unbounded. The replayed batch performs only the calls the signer already authorized, so the effect is not an unauthorized action but a self-authorized batch executing later than a signer reasoning by EOA analogy would expect.

Consider documenting that a signed batch consumes its nonce only on success, and recommending short, non-zero deadlines alongside the self-service `useNonce` cancellation path to invalidate the signature of a batch that has failed.

Update: Resolved in [pull request #1](#). The team stated:

`_verifySignedBatch`, the `HermesV1` contract docstring and the README now state that a signed batch spends its nonce whenever the transaction does not revert, a batch ended early by a break policy spends it too, together with the two mitigations: sign a short, non-zero `deadline`, and retire a failed batch's signature with `useNonce()`.

N-03 Zero-Address Targets Are Signed Verbatim but Executed as Self-Calls

The `execute` function of the `HermesDelegateV1` contract supports a signed batch mode in which authorization is provided by an EIP-712 signature verified in `_verifySignedBatch`. The signed digest is built by `_hashCalls`, which includes each `Call.target` verbatim. However, during execution, `_executeBatch` rewrites any target equal to `address(0)` to `address(this)`, following ERC-7821. As a result, the target that is signed can differ from the target that is executed, and two distinct signable representations of the same self-call exist.

This discrepancy does not bypass authorization, as the calldata bytes remain covered by the signature. Nevertheless, it reduces off-chain review clarity and increases phishing risk. In particular, a relay could request a signature over a call whose `data` encodes a nested `execute` batch while signing tooling renders the target literally as `0x000...000` and fails to decode the nested payload, even though the call is executed against the account itself.

Consider canonicalizing `Call.target` before hashing by normalizing `address(0)` to `address(this)`, or rejecting zero-address targets in the signed batch path. In addition, consider emitting an explicit event whenever a zero-address target is rewritten so that self-calls remain observable for monitoring purposes.

Update: Resolved in [pull request #2](#). The team stated:

`_hashCalls` now canonicalizes a `Call.target` of `address(0)` to `address(this)` before hashing, the same rewrite `_executeBatch` applies at execution time. `_hashCalls` becomes `view`, and a signature produced over a literal zero target no longer validates.

Tests: the two gasless-relay compositions sign the account's real address and submit the zero-address shorthand; two new cases pin both directions.

N-04 Non-Explicit Imports Are Used

The use of non-explicit imports in a codebase can decrease code clarity and may create naming conflicts between locally defined and imported variables. This is particularly relevant when multiple contracts exist within the same Solidity file or when inheritance chains are long.

Within `HermesV1.sol`, the `IHermesNonce.sol` interface is imported globally, whereas the rest of the codebase already uses named imports.

Consider updating `HermesV1.sol` to use the named import syntax (`import {A, B, C} from "X"`) so that the imported contracts or interfaces are explicitly declared, consistent with the rest of the codebase.

Update: Resolved in [pull request #3](#). The team stated:

`HermesV1.sol` now imports `IHermesNonce` with the named import syntax.

N-05 Missing Named Parameters in Mapping

Since [Solidity 0.8.18](#), mappings can include named parameters declared in the form `mapping(KeyType KeyName? => ValueType ValueName?)` to provide more clarity about their purpose.

Within `HermesV1.sol`, the `_nonces` state variable is a mapping without named parameters.

Consider adding named parameters to mappings in order to improve the readability and maintainability of the codebase.

Update: Resolved in [pull request #4](#). The team stated:

`_nonces` is now declared as `mapping(address account => uint256 nonce)`
`private _nonces;`

N-06 Missing Security Contact

Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, reducing the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract

incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

Throughout the codebase, multiple instances of contracts not having a security contact were identified:

- The [HermesBase](#) abstract contract
- The [IERC7779](#) interface
- The [IHermesNonce](#) interface
- The [HermesDelegateV1](#) contract
- The [HermesV1](#) contract

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the [@custom:security-contact](#) convention is recommended as it has been adopted by the [OpenZeppelin Wizard](#).

Update: Resolved in [pull request #5](#). The team stated:

[@custom:security-contact bugs@tonkeeper.com](#) added to [HermesBase](#), [HermesDelegateV1](#), [HermesV1](#), [IERC7779](#) and [IHermesNonce](#).

N-07 Magic Numbers In The Code

Within [HermesDelegateV1.sol](#), there are occurrences of literal values with unexplained meanings:

- The [2](#) literal, representing the bit width of each per-call policy slot within the packed [policies](#) word.
- The [3](#) literal, representing the two-bit mask used to extract a single policy value.

Consider defining and using [constant](#) variables instead of these literal values to improve the readability of the codebase.

Update: Resolved in [pull request #7](#). The team stated:

The [2](#) and [3](#) literals in [_executeBatch](#) are now [POLICY_BITS](#) and [POLICY_MASK](#), and [MAX_TRY_CALLS](#) is derived as [MODE_PAYLOAD_BITS / POLICY_BITS](#) instead of the hardcoded 88.

N-08 Inconsistent Use of Named Return Variables

Within `HermesDelegateV1.sol`, there are multiple instances of inconsistently used or unused named return variables:

- The `_decodeExecutionMode` function declares the named return variables `id`, `tryExec`, and `payload`, but exits exclusively through explicit `return` statements. The `id` variable is never assigned, whereas `tryExec` and `payload` are assigned in the body and then returned explicitly regardless.
- The `eip712Domain` function declares seven named return variables that are never assigned, returning all of its values through an `explicit return statement` instead.

Consider either assigning and implicitly returning the named return variables or removing them in favor of unnamed return types, and applying the chosen style consistently throughout the codebase to improve its readability and maintainability.

Update: Resolved in [pull request #8](#). The team stated:

`eip712Domain` assigns its seven named returns instead of shadowing them with an explicit tuple. The ABI output names are unchanged.

- *`_decodeExecutionMode` drops its names in favour of returns (`ModeId`, `bool`, `uint176`), keeping the explicit tuple at every exit.*

N-09 Signed Batch Execution Is Not Observable From Event Logs

When a batch is executed through the `opData` path, the `execute` function of `HermesDelegateV1` verifies the signature and consumes the account's nonce in `_verifySignedBatch` without emitting any event. The only events emitted by the contract, `ERC7579TryExecuteFail` and `BatchInterrupted`, occur exclusively in try mode upon a call failure or an interruption, so a successful batch produces no logs from the account itself.

Unlike the ERC-4337 path, whose execution is observable through the EntryPoint's `UserOperationEvent`, an `opData` batch is signed by the delegated EOA but submitted by an arbitrary relayer, and comparable relay mechanisms (for example, ERC-2771 forwarders and Safe accounts) emit a canonical execution event for this reason. As a result, off-chain relayers, indexers, and monitoring systems cannot learn from a log subscription that a given signed batch was executed or that a given nonce was consumed, and must instead poll `nonce0f` or

trace transaction calldata, for instance to detect that a distributed signature was executed by a third party.

Consider emitting an event such as `Executed(uint256 indexed nonce, bytes32 mode, uint256 callCount)` in the `opData` branch of `execute` so that signed-batch execution and nonce consumption are observable from the account's own logs.

Update: Acknowledged, not resolved. The team stated:

Won't fix; the behaviour is intended.

We are aware this differs from the ERC-4337 path, where the EntryPoint emits an event for every operation. Here the logs answer a narrower question: which calls did not execute, and where the batch stopped short. That is what `CallFailed(index)` and `BatchInterrupted(index)` record. A transaction that did not revert is a success in itself, so there is nothing further to announce.

Tracking what a successful batch did belongs to the contracts it calls, e.g. ERC-20 transfers logs two `Transfer` events.

A consumer restricted to log subscriptions cannot filter by transaction recipient and has to poll `nonceOf` instead; we accept that

N-10 Misleading, Incomplete, or Missing Documentation

Throughout the codebase, multiple instances of missing docstrings were identified:

- In `HermesBase.sol`, the `ERC7779Storage struct` and the `supportsInterface`, `onERC721Received`, `onERC1155Received`, `onERC1155BatchReceived`, and `accountStorageBases` functions lack docstrings.
- In `HermesDelegateV1.sol`, the `HERMES_NAME_HASH` and `VERSION_HASH` constants, the `MODE_SELECTOR_NO_OPDATA` and `MODE_SELECTOR_OPDATA` constants, the `onlyEntryPoint` modifier, and the `receive` function lack docstrings.
- In `IERC7779.sol`, the `IERC7779 interface` lacks a docstring. In addition, the comments on the `accountId` and `accountStorageBases` functions are written as regular block comments, which are not recognized by NatSpec tooling.
- In `IHermesNonce.sol`, the `IHermesNonce interface` and its `useNonce` function lack docstrings.

In addition, multiple instances of incomplete docstrings were identified, where a docstring exists but does not document all of the function's parameters and return values (in several cases, the information appears only in prose, without the `@param`/`@return` tags recognized by NatSpec tooling):

- In `HermesBase.sol`, the `tokensReceived` function's parameters, the `onTokenTransfer` function's parameters and return value, and the `accountId` function's return value are not documented.
- In `HermesDelegateV1.sol`, the parameters and/or return values of the `isSupportedEntryPoint`, `accountId`, `supportsInterface`, `validateUserOp`, `supportsExecutionMode`, `isValidSignature`, and `eip712Domain` functions are not documented.

In addition, multiple instances of misleading documentation were identified:

- In `HermesV1.sol`, the contract-level docstring refers to `HermesDelegateV1.executeWithSignature` as the signature-based execution path and later describes the lifecycle of `a pending executeWithSignature signature`. However, no such function exists, as signed execution is performed through the `opData` mode of the `execute` function, which the `HermesDelegateV1` contract's `own docstring` states to have replaced `executeWithSignature`.
- In `HermesDelegateV1.sol`, the `comment on the ERC7739_SUPPORT_HASH constant` describes the ERC-7739 detection probe as `isValidSignature(this, "")`. Since `this` denotes the contract itself in Solidity, the comment is misleading, as the probe's first argument is the `0x7739...7739` constant hash rather than the account address.
- In `HermesDelegateV1.sol`, the docstrings of the `execute` and `verifySignedBatch` functions state that "the nonce is consumed before any external call". This is imprecise, as the nonce consumption through `manager.useNonce()` is itself an external call, and the intended meaning is that it happens before any of the batch's calls.
- In `HermesDelegateV1.sol`, the `contract-level docstring` describes the execution modes as replacing the `execute`, `executeBatch`, `executeUserOp`, `executeUserOps`, and `executeWithSignature` functions. As these functions never existed in the codebase, readers without historical context may search for them in vain.
- In `README.md`, the `documentation of the trusted EntryPoints` states that "`entryPoint()` reports v0.9 for discovery". However, no `entryPoint()` function exists in the codebase, and a call to that selector is silently absorbed by the empty `fallback` function, returning no data. As a result, SDKs and indexing tooling that read this conventional ERC-4337 getter for EntryPoint discovery will fail to decode the empty

return value, and discovery is only possible through the non-standard `isSupportedEntryPoint` function. Consider either implementing the documented `entryPoint()` getter (returning the v0.9 EntryPoint address) or updating the README to describe EntryPoint discovery through `isSupportedEntryPoint`.

Consider thoroughly documenting all functions (along with their parameters and return values) that are part of any contract's public API, as well as any functions implementing sensitive functionality, and updating the misleading documentation to accurately reflect the current implementation. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in [pull request #9](#). The team stated:

Misleading statements corrected:

- `HermesDelegateV1.executeWithSignature`, which does not exist, it is the `opData` mode of `execute`.
- The contract docstring describing the modes as "replacing" functions that never existed here.
- The `ERC7739_SUPPORT_HASH` probe written as `isValidSignature(this, "")`; its first argument is the `0x7739...7739` constant.
- "the nonce is consumed before any external call", `manager.useNonce()` is itself an external call; the guarantee is that it precedes the batch's calls.
- The README's `entryPoint()` getter, which does not exist. `isSupportedEntryPoint` is now documented as the discovery surface. The `Execute` typehash in the flow table also omitted `deadline`.

Fourteen missing docstrings added, covering the ERC-7779 storage struct and slot, the token-receiver callbacks, `supportsInterface`, `accountStorageBases`, the domain and mode-selector constants, `onlyEntryPoint`, `receive`, and both interfaces. IERC7779's block comments are now NatSpec-recognized.

Ten functions now carry `@PARAM` and `@return` tags: `tokensReceived`, `onTokenTransfer`, `accountId`, `isSupportedEntryPoint`, `supportsInterface`, `validateUserOp`, `supportsExecutionMode`, `isValidSignature` and `eip712Domain`.

N-11 Native Value Attached to a Signed Batch Is Silently Spendable on the Relayed Path

The `execute` function is `payable`, and its signature-authorized `opData` branch neither rejects nor bounds `msg.value`. The signed digest is built from `EXECUTE_TYPEHASH`, which commits only to the mode, calls, nonce, and deadline, so the native value attached to the transaction lies outside what the account holder authorizes. Because an EIP-7702 account's balance is simply the underlying EOA's balance, any value a submitter attaches is credited to the account and becomes immediately spendable through the batch's [per-call value transfers](#).

This path is intended to be self-funding, and no standard flow attaches value to it: the `EntryPoint` always invokes `execute` with zero value, a self-call carrying value is a no-op round trip, and ERC-7821 requires only that `execute` be `payable` while leaving the treatment of `msg.value` to the implementation. A submitter that nonetheless attaches value, for instance a relayer led to believe it must fund the batch's transfers, has that value consumed by the signed calls, with the loss falling on the submitter. A submitter that always attaches zero value is unaffected, since a batch spending more than the account holds simply reverts.

Consider reverting on the `opData` branch when `msg.value` is non-zero, while keeping `execute` itself `payable` for ERC-7821 conformance, so that the relayed path is provably self-funding. If value-carrying relayed batches are ever desired, consider instead binding `msg.value` into the `Execute` typehash so the signer authorizes the exact amount, noting that this makes the amount explicit but does not by itself shield a submitter that chooses to fund a malicious batch.

Update: Acknowledged, not resolved. The team stated:

Won't fix; the behaviour is intended.

`msg.value` on the `opData` path is a deliberate capability: it lets a submitter fund the batch's native transfers atomically, sponsoring the transferred amount, not just the gas. The submitter sets it explicitly and is the only party exposed, so a contract-level guard would protect a party against a parameter it chooses itself while removing the sponsorship flow. ERC-7821 requires only that `execute` be `payable` and leaves the treatment of `msg.value` to the implementation.

Binding `msg.value` into the `Execute` is a field of the submitter's own transaction, signed by the party that pays it, while `Call.value` is signed by the account holder that spends it.

Conclusion

Hermes is a deliberately minimal smart-account system for EIP-7702-delegated EOAs, consisting of the HermesDelegateV1 implementation contract and the HermesV1 singleton nonce manager. Several design choices materially reduce the system's attack surface. The delegate holds no mutable storage of its own, every execution flow converges on a single ERC-7821 entry point, the delegated EOA's key remains the sole authority over the account, and replay protection is confined to a per-account counter inside an ownerless and non-upgradeable singleton. Consistent with this restraint, the audit did not identify any critical-, high-, or medium-severity issues.

The findings that were identified are of low severity or informational in nature, and they cluster around three themes. The first theme is the lifecycle of signed batches, where a signature whose execution reverts on-chain remains replayable because its nonce consumption is rolled back, where re-delegating the account does not reliably invalidate outstanding signatures, and where a signed batch that itself interacts with the nonce manager can unintentionally invalidate other pending signatures. The second theme is the novel try-mode policy encoding, in which a call marked as required can be silently skipped when a preceding call triggers a break policy, and in which a malicious call target can force an otherwise fault-tolerant batch to revert by returning oversized data. The third theme concerns integration behavior at the account's boundaries, including the rejection of raw ECDSA, EIP-191, and EIP-2098 signatures on the account's signing surfaces, and the treatment of zero-address targets as self-calls. None of these issues undermines the core authorization model, but each represents behavior that wallet developers and relayers interacting with Hermes should understand precisely.

The codebase was found to be well-written, extensively documented, and accompanied by a thorough test suite. The in-line documentation is notable for explaining not only what each function does but also the trust assumptions and standards-compliance reasoning behind it. To further strengthen the system, consider expanding the event coverage of the signature-based execution path so that signed batch executions are observable from event logs, documenting the exact set of accepted signature encodings for integrators, and clarifying the intended semantics of the try-mode policy bits in user-facing documentation. Given that the nonce manager's address is a security-critical deployment parameter that every delegate pins immutably, we also recommend establishing a verified and well-publicized deployment process for the singleton.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.